

Хуки репозитория git.sbis.ru



Более подробная информация по хукам размещена на GitLab:

- [Описание хуков](#)
- [Администрирование хуков](#)

Хук Git – это скрипт, который автоматически запускается каждый раз, когда в репозитории Git происходит определенное событие. С помощью хуков можно конфигурировать внутреннее поведение Git и запускать настраиваемые действия в ключевые моменты жизненного цикла разработки.

Обычно хуки Git используют, чтобы поддерживать политику коммитов, изменять среду проекта в зависимости от состояния репозитория и внедрять рабочие процессы непрерывной интеграции. Однако эти скрипты можно настраивать с учетом любых предпочтений, поэтому они помогут автоматизировать или оптимизировать любую сторону процесса разработки.

Все хуки git.sbis.ru хранятся в репозитории [git-hooks](#). В папке **git-hooks/hookfiles/rules** хранятся проверочные файлы – python-файлы, которые должны экспортировать метод `run(commitInfo, logger)`, где:

- **commitInfo** – это объект класса [CommitInfo](#),
- **logger** – объект стандартного класса `Logger`.

Проверочные файлы запускаются последовательно в скрипте **hookfiles/module_check_main.py**. Отдельно регистрировать такие файлы не нужно, т.к. запускаются все python-файлы из папки **rules**.

Логирование выполняется в файл на сервере GitLab, при этом имя файла лога будет соответствовать имени репозитория.

Основные требования к проверочным файлам требование к проверкам:

- не должно быть явного вывода в **stdout**.
- пользоваться **print()** нельзя, т.к. если команда **git push** выполняется в Windows, то переменная **sys.stdout.encoding** будет равна строке 'ANSI_X3.4-1968'. Как результат – скрипт будет падать на первом неправильном символе (кириллическом, например). Если **git push** выполняется из Linux, то **sys.stdout.encoding** будет равна строке 'utf8'.
- при запуске хука из GitLab первая строка (до переноса строки) выводится как ошибка. Остальной вывод будет впустую. Приватных проверок нет. Если какой-то проверочный файл нужно применять только к одному репозиторию, то необходима валидация **commitInfo.repositoryName** с последующим выходом.

Для тестирования хуков используется специальный ресурс <http://dev-git.sbis.ru>.

Класс CommitInfo

Инкапсулирует работу с Git.

Поля

- **commit_info.revision_before** – ревизия до.
Пример: 95b4a82e2d7453d54c3386541591a42c383a97a8
- **commit_info.revision_after** – ревизия после.
Пример: 95b4a82e2d7453d54c3386541591a42c383a97a8
- **commit_info.branch_name** – полное имя ветки.
Пример: refs/heads/3.18.110/bugfix/TEST
- **repositoryName** – короткое имя репозитория (group/project).
Пример: "sbis-ci/test-hook" для репозитория "git@fix-git.sbis.ru:sbis-ci/test-hook.git"
- **commitAuthor** – автор коммита.
- **changedFiles** – список изменённых файлов (пути относительно корня репозитория).
- **runFromGitlab** – хук запущен из GitLab. Для него нужен особый вывод ошибок.

Методы

- **writeError(self, ruleName, message, filePath="")** – вывести ошибку пользователю. Все ошибки будут выведены в конце работы хука.
 - **ruleName** – имя проверки,
 - **message** – текст ошибки,
 - **filePath** – путь до файла с ошибкой (опционально).
- **checkBranchVersion(self, version)** – проверка имени ветки по tensor flow. Нужно для внедрения проверочного файла с конкретной версии, чтобы не мешать выпуску более старых версий, например, хотфиксов.
- **getFileContent(self, filePath)** – получить содержимое файла как массив байтов. После этого массив декодируется в utf8. Получить можно только файлы, указанные в поле CommitInfo.changedFiles.

Прикладная часть

Создание хука включает в себя следующие этапы:

- [создание скрипта хука](#)
- (опционально) [внесение изменений в окружение сервера GitLab](#) – установка дополнительных модулей, программ, создание общих ресурсов и т.д.
- [тестирование работы хука](#) на разных версиях GitLab
- [создание документации](#)
- [передача хука в сборку](#)

Создание скрипта хука

В компании "Тензор" используются хуки [pre-receive](#), которые запускаются на сервере после выполнения команды **git push**, непосредственно перед отправкой изменения в хранилище. По коду выхода хука GitLab либо применяет изменение, либо отклоняет коммит.

Для упрощения разработки хуков используется фреймворк, который выполняет типовые операции по работе с коммитом. Состав фреймворка:

- [module_check_main.py](#) – скрипт инициализирует среду выполнения хуков и занимается их непосредственным запуском и обработкой полученной информации.
- [commitinfo.py](#) – содержит класс CommitInfo, который получает всю необходимую информацию о коммите и позволяет выполнить типовые операции с коммитом, например, получение содержимого файлов.
- [hook_helpers.py](#) – дополнительные функции, которые могут быть полезны при разработке собственного хука.

Правила

- хук может быть написан на любом языке, но для запуска необходим стартовый python-скрипт (.py), расположенный в папке [git-hooks/hookfiles/rules](#) и содержащий функцию **def run(commit_info, logger):**, которую будет вызывать фреймворк.
- хук не должен содержать команду выхода: при ненулевом коде коммит будет отклонен, а при нулевом – не запустятся остальные хуки
- чтобы хук не проверял код дважды (во время коммита ветки/изменений и повторно в Merge request), добавьте в начале:

```
if commit_info.is_accept_merge_request: return
```

Внесение изменений в окружение сервера GitLab

Если для работы хука необходимы дополнительные модули Python, установка программ и любая другая модификация окружения – оформите поручение с перечнем необходимого ПО в отдел [Мониторинга](#).

Тестирование работы хука

Для проверки работы хука и его окружения создана сборка [gitlab_dev](#). Она позволяет развернуть на [dev-git.sbis.ru](#) хуки из вашей ветки.

Также можно напрямую отредактировать файлы. На [dev-git.sbis.ru](#) доступны следующие ресурсы (открываются на сетевом диске):

- [\\dev-git.sbis.ru\hooks_*](#) – каталоги, содержащие хуки тестовых репозиториев.
- [\\dev-git.sbis.ru\logs](#) – каталог, содержащий логи хуков.
- [\\dev-git.sbis.ru\tmp](#) – каталог для временных файлов, используемый некоторыми хуками.

Используя эту сборку и указанные ресурсы, вы можете подготовить и выложить окружение и сами хуки, попробовать оформить коммит/Merge request файлов или веток в тестовый репозиторий, а также отслеживать поведение хука.

Документация

К каждому хуку должна быть оформлена краткая документация, отвечающая на вопросы:

- Заказчик хука
- Автор хука
- Назначение хука
- Состав (если несколько скриптов, конфигураций, папок)
- Алгоритм работы
- Примеры срабатываний (скриншоты)
- Как проверить работу

Все ссылки на документацию для каждого хука размещаются [здесь](#).

Пример оформления документации для хука: [check_py_and_orx_syntax](#).

Передача хука в сборку

Для внедрения хука в боевой гитлаб оформите поручение в отдел [мониторинга](#) со следующим обязательным содержанием:

- по хукам: ссылка на Merge request из вашей ветки в master.
- ссылка на созданную документацию по хуку.
- ссылка на новость отдела разработки о новом хуке (если хук общий для всех репозиториев).
- контрольные файлы:
 - good ("правильный файл") – должен пройти хук,
 - bad ("сломанный файл") – должен быть отброшен.

Затем проводится проверка кода, при необходимости – ревью с привлечением сторонних программистов, проверка работоспособности хука и заполнения документации. После завершения этих этапов публикуется новость и хук устанавливается на требуемые репозитории.